

Day 5

TASK 1

```
class GreetingThread extends Thread {

    public GreetingThread(String name) {
        this.name = name;
    }

    @Override
    public void run() {
        try {
            String name = Thread.currentThread().getName();

            if (name.equals("Morning")) {
                System.out.println("Good Morning");
            } else if (name.equals("Afternoon")) {
                System.out.println("Good Afternoon");
            } else if (name.equals("Evening")) {
                System.out.println("Good Evening");
            }

        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}

public class Day5_GoodMoringing{
    public static void main(String[] args) {

        GreetingThread t1 = new GreetingThread("Morning");
        GreetingThread t2 = new GreetingThread("Afternoon");
        GreetingThread t3 = new GreetingThread("Evening");

        t1.start();
        t2.start();
        t3.start();
    }
}
```

TASK 2

```

class GreetingTask implements Runnable {
    private String message;
    private int sleepTime;

    public GreetingTask(String message, int sleepTime) {
        this.message = message;
        this.sleepTime = sleepTime;
    }

    public void run() {
        try {
            Thread.sleep(sleepTime);
            System.out.println(message);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

public class Day5_GoodMoringing {
    public static void main(String[] args) {

        Thread t1 = new Thread(new GreetingTask("Good Morning", 2000));
        Thread t2 = new Thread(new GreetingTask("Good Afternoon", 1000));
        Thread t3 = new Thread(new GreetingTask("Good Evening", 3000));

        t1.start();
        t2.start();
        t3.start();

        try {
            t1.join();
            t2.join();
            t3.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("All threads completed.");
    }
}

```

TASK 3

```

class GreetingThread extends Thread {

    public GreetingThread(String name) {
        super(name);
    }

    @Override
    public void run() {
        try {
            String threadName = Thread.currentThread().getName();

```

```

        if (threadName.equals("Morning")) {
            Thread.sleep(2000);
            System.out.println(threadName + " : Good Morning");
        } else if (threadName.equals("Afternoon")) {
            Thread.sleep(1000);
            System.out.println(threadName + " : Good Afternoon");
        } else if (threadName.equals("Evening")) {
            Thread.sleep(3000);
            System.out.println(threadName + " : Good Evening");
        }
    }
} catch (InterruptedException e) {
    System.out.println(e);
}
}
}

public class Day5_GoodMorning {
    public static void main(String[] args) {

        GreetingThread morning = new GreetingThread("Morning");
        GreetingThread afternoon = new GreetingThread("Afternoon");
        GreetingThread evening = new GreetingThread("Evening");

        morning.start();
        afternoon.start();
        evening.start();

        try {
            morning.join();
            afternoon.join();
            evening.join();
        } catch (InterruptedException e) {
            System.out.println(e);
        }

        System.out.println("All threads completed.");
    }
}

```

TASK 4

```

class GreetingThread extends Thread {

    public GreetingThread(String name) {
        super(name);
    }

    @Override
    public void run() {

```

```

        System.out.println(getName() + " is running");
    }
}

public class Day5_GoodMoringing {
    public static void main(String[] args) {

        GreetingThread t1 = new GreetingThread("Good Morning");
        GreetingThread t2 = new GreetingThread("Good Afternoon");
        GreetingThread t3 = new GreetingThread("Good Evening");

        t1.setPriority(Thread.MAX_PRIORITY); // 10
        t2.setPriority(Thread.NORM_PRIORITY); // 5
        t3.setPriority(Thread.MIN_PRIORITY); // 1

        t1.start();
        t2.start();
        t3.start();

        System.out.println(t1.getName() + " Priority: " + t1.getPriority());
        System.out.println(t2.getName() + " Priority: " + t2.getPriority());
        System.out.println(t3.getName() + " Priority: " + t3.getPriority());
    }
}

```

TASK 5

Wap to run threads in a loop.. of 1 to 20

Print the dumpstack..

```

class MyRunnable implements Runnable {

    @Override
    public void run() {
        for (int i = 1; i <= 20; i++) {
            System.out.println("Value: " + i);
            Thread.dumpStack();
        }
    }
}

public class Day5_ThreadLoop {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyRunnable(), "Thread-1");
        t1.start();
    }
}

```

TASK 6

Check for isalive status of a thread..

```

class MyRunnable implements Runnable {
    public void run() {
        for (int i = 1; i <= 20; i++) {

```

```

        System.out.println("Value: " + i);
        Thread.dumpStack();
    }
}

}

public class Day5_ThreadLoop {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyRunnable(), "Thread-1");
        System.out.println("Before start(): " + t1.isAlive());
        t1.start();
        System.out.println("After start(): " + t1.isAlive());
        try {
            t1.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("After completion: " + t1.isAlive());
    }
}

```

TASK 7

synchronized

```

class GreetingThread1 extends Thread {

    public GreetingThread1(String name) {
        super(name);
    }

    @Override
    public void run() {
        System.out.println(getName() + " is running");
    }
}

public class Day5_GoodMoringing {
    public static void main(String[] args) {

        GreetingThread t1 = new GreetingThread("Good Morning");
        GreetingThread t2 = new GreetingThread("Good Afternoon");
        GreetingThread t3 = new GreetingThread("Good Evening");

        t1.setPriority(Thread.MAX_PRIORITY); // 10
        t2.setPriority(Thread.NORM_PRIORITY); // 5
        t3.setPriority(Thread.MIN_PRIORITY); // 1

        t1.start();
        t2.start();
        t3.start();

        System.out.println(t1.getName() + " Priority: " + t1.getPriority());
        System.out.println(t2.getName() + " Priority: " + t2.getPriority());
        System.out.println(t3.getName() + " Priority: " + t3.getPriority());
    }
}

```

```
}  
}
```

TASK 8

```
public class Day5_exception{  
    public static void main(String[] args) {  
        try {  
            int result = 10 / 0;  
        } catch (ArithmeticException e) {  
            System.out.println("zero division error ");  
        }  
        try {  
            int arr[] = {1, 2, 3};  
            System.out.println(arr[5]);  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("array index not found");  
        }  
        finally {  
            System.out.println("Finally Block Executed");  
        }  
    }  
}
```

TASK 9

Lambda function

```
import java.util.ArrayList;  
import java.util.Arrays;  
import java.util.List;  
  
public class Day5_Lambda {  
  
    public static void main(String[] args) {  
        Runnable r1 = new Runnable() {  
            @Override  
            public void run() {  
                System.out.println("hello");  
            }  
        };  
        Runnable r2 = () -> System.out.println("hello2");  
        r1.run();  
        r2.run();  
        List<String> names = new ArrayList<>(  
            Arrays.asList("monika", "likitha", "thanu")  
        );  
        names.forEach(name -> System.out.println("Name : " + name));  
    }  
}
```

Task 10

Lambda Add and Multiply

```

interface Calculator1 {
    int calculate(int a, int b, int c);
}

public class Day5_LambdaAdd {
    public static void main(String[] args) {
        Calculator1 add = (a, b, c) -> a + b + c;
        Calculator1 multiply = (a, b, c) -> a * b * c;
        System.out.println("Addition: " + add.calculate(10, 20, 30));
        System.out.println("Multiplication: " + multiply.calculate(10, 20, 30));
    }
}

```

TASK 11

FUNCTION , PREDICATE, SUPPLIER

```

import java.util.function.Function;
import java.util.function.Predicate;
import java.util.function.Supplier;

@FunctionalInterface
interface Calculator {
    int calculate(int a, int b, int c);
}

public class Day5_functionalinterface {

    public static void main(String[] args) {

        Calculator1 add = (a, b, c) -> a + b + c;
        Calculator1 multiply = (a, b, c) -> a * b * c;
        System.out.println("Addition: " + add.calculate(10, 20, 30));
        System.out.println("Multiplication: " + multiply.calculate(10, 20, 30));

        Function<Integer, Integer> square = x -> x * x;
        System.out.println("Square of 5: " + square.apply(5));

        Predicate<Integer> isEven = x -> x % 2 == 0;
        System.out.println("Is 10 Even? " + isEven.test(10));
        System.out.println("Is 11 Even? " + isEven.test(11));

        Supplier<String> message = () -> "Hello Monika!";
        System.out.println("Supplier Value: " + message.get());
    }
}

```